

OpenChoreo Developer Platform: Developer-Side UI Reference

*Prepared for Project 702 (PR-Driven Self-Service GitOps). Source: [openchoreo.dev docs v1.3.x](#) (OpenChoreo v1.3.1) and the Backstage-powered portal pages. Where a statement is my own design inference rather than something the docs say, it is marked **[Inference]**.*

Limits of this document. I read the documentation text; I could not open the running portal or view its screenshots. Layout details (exact button positions, colors) are therefore not covered. Component names, tabs, routes and behaviours are taken from the docs.

1. What OpenChoreo is

OpenChoreo is an open-source, modular Internal Developer Platform for Kubernetes. It bundles abstractions for platform and application teams, guardrails, a Backstage-powered developer portal, CI/CD, GitOps and observability. It was originally developed by WSO2 as the successor to the WSO2 Choreo SaaS platform and is now a CNCF project. Its stated principle is that it does not hide Kubernetes: platform APIs are Kubernetes-native and reconciled by controllers in the control plane.

Multi-plane architecture

Plane	Role
Control Plane	The "brain". Turns intent declared in APIs into Kubernetes resources, enforces policy, reconciles drift. Exposed via CLI, API, MCP and UI.
Data Plane(s)	Runs application workloads. Logical environments, project and tenant isolation, gateways, network security.
Workflow Plane(s)	Runs builds and CI, plus general automation workflows (for example IaC). Build execution uses Cloud Native Buildpacks and Argo Workflows.
Observability Plane(s)	Ingests and serves logs, metrics, traces and alerts.

Plane	Role
Experience plane	The portal and CLI/MCP surfaces, with modules for tool integrations.

The portal's guiding idea: intent flows down (developer action to Kubernetes resources), reality flows up (runtime state aggregated back to the portal).

2. Developer abstractions (the vocabulary the UI is built on)

The UI is a view over these resources. Any UI change you make should keep this vocabulary.

Abstraction	Meaning for a developer	Backstage entity kind
Namespace	Ownership and isolation boundary for teams, users and resources.	Domain
Project	Bounded context grouping related components. Becomes an isolated deployment unit (dedicated namespaces, network boundaries). References a ProjectType and a DeploymentPipeline.	System
Component	A deployable unit (service, web app, job). Combines a ComponentType reference, parameters, traits and an optional build Workflow.	Component
Workload	The runtime contract of a component: container, endpoints (HTTP, gRPC, GraphQL, Websocket, TCP, UDP) and dependencies.	part of Component
Resource	A managed-infrastructure dependency (database, queue, cache) declared against a ResourceType. Project-bound.	Resource
WorkflowRun	One execution of a Workflow (for example a build) with specific parameter values.	shown in Build / Runs tabs

Release model (central to your project)

This is the most important part for a PR-driven design.

1. A **ComponentRelease** is an immutable snapshot of the Component (and its type and parameters) taken at release time.
2. A **ReleaseBinding** pins a ComponentRelease to one **Environment**. The same release is deployed to dev, staging and prod; only per-environment overrides differ.
3. **Promotion** means creating or updating the ReleaseBinding for the next environment in the **DeploymentPipeline**. The release itself never changes.
4. Projects follow the same pattern: **ProjectRelease** plus one **ProjectReleaseBinding** per environment.
5. Resources follow it too: **ResourceRelease** plus **ResourceReleaseBinding**.

So in GitOps terms, the files a developer-driven PR should produce are ReleaseBinding manifests (promotion and override changes), ComponentRelease and Workload manifests (new versions), and Component manifests (new components).

3. The developer portal: structure and navigation

The portal is Backstage with OpenChoreo plugins. Default install:

`http://openchoreo.localhost:8080/` in the quick-start (local k3d), signed in through the OpenChoreo identity provider (OIDC). If the cluster runs with auth off, it falls through to guest sign-in.

3.1 Top-level areas

- **Catalog / Home.** Namespaces, projects, components, endpoints, resources and dependencies appear as catalog entities that carry runtime meaning (a project is an isolation boundary, a dependency is enforced traffic flow).
- **Create.** The Backstage scaffolder page. Default OpenChoreo templates cover creating projects, components, resources, namespaces, environments, deployment pipelines, component types, resource types, traits and workflows (and their cluster-scoped variants). Custom templates appear under an **Other Templates** section.
- **Import to Catalog.** The Backstage wizard at `/catalog-import` (Analyze, then Import).
- **Workflows page.** A standalone `/workflows` page for generic workflows (optional workflows plugin pack).

- **Platform view.** One view of namespaces, data planes, environments, component types and traits (mostly platform-engineer facing).
- **Access control.** RBAC management, available when authorization is enabled.

3.2 Entity pages and tabs (what a developer sees)

Domain (Namespace) entity

Tab	Contents
Overview	Cards: namespace projects, namespace resources.
Definition	The raw OpenChoreo resource manifest.

System (Project) entity

Tab	Contents	Package
Overview	Project components card, deployment pipeline card, cell topology card.	core
Definition	Raw manifest.	core
Cell Diagram	Live architecture graph of components, endpoints, dependencies, reflecting permitted traffic.	core
Logs	Project-scoped runtime logs.	observability
Traces	Distributed traces.	observability
Incidents	Project incidents.	observability
RCA Reports	Reports from the built-in root-cause-analysis (SRE) agent.	observability
Cost Analysis	FinOps agent reports.	observability

Component entity (the developer's main workspace)

Tab	Route	Contents	Package
Overview	built-in	Deployment status card, runtime health card, Deployments widget linking to Deploy.	core
Definition	<code>/definition</code>	Raw Component manifest.	core
Build	<code>/workflows</code>	Workflow runs and a trigger form (dynamic parameter forms via <code>@rjsf</code>).	ci
Deploy	<code>/environments</code>	Per-environment cards: deploy, promote, overrides, status.	core
Logs	<code>/runtime-logs</code>	Component runtime logs.	observability
Events	<code>/runtime-events</code>	Kubernetes/runtime events.	observability
Metrics	<code>/metrics</code>	Metrics charts (recharts).	observability
Alerts	<code>/alerts</code>	Alerts.	observability
Wirelogs	<code>/wirelogs</code>	Service-mesh wire-level logs.	observability

Other entity kinds (Environment, DataPlane, WorkflowPlane, ObservabilityPlane, DeploymentPipeline, Workflow, ClusterWorkflow) have their own overview cards (30+ cards across kinds) and a **Runs** tab on Workflow entities.

4. The Deploy tab in detail (the screen Project 702 changes most)

The Deploy tab renders the `Environments` component at `/environments`. It reads the entity's OpenChoreo annotations (for example `openchoreo.io/component-name`) to know which project and component to query, and calls the OpenChoreo API through the Backstage backend proxy at `/api/openchoreo/*`. Components imported from a hand-written `catalog-info.yaml` without those annotations show an "OpenChoreo metadata not found" empty state.

4.1 What it shows

One **environment card** per environment in the deployment pipeline (for example development, staging, production). Each card shows:

- Deployment status: **Ready**, **NotReady** or **Failed**
- Last deployed timestamp
- Container image reference
- Release name (for example `my-service-5d7f658d9c`)
- Endpoint URLs
- A **View K8s Artifacts** action that opens the full resource tree: Deployments, Pods, Services, HTTPRoutes

4.2 What a developer can do from it

Action	How in the UI	What happens underneath
First deploy	First environment card. With <code>autoDeploy: true</code> it deploys automatically when the Component and Workload are created.	Control plane creates the ReleaseBinding for the root environment.
Promote	Promote button on the deployed environment's card.	Promotes to the next environment in the pipeline via the API.
Override configuration	Gear icon on an environment card header opens an Overrides page with tabs Workload (env vars, file mounts, endpoints), Component (ComponentType schema parameters) and Traits (per trait instance).	Writes <code>workloadOverrides</code> , <code>componentTypeEnvironmentConfigs</code> , <code>traitEnvironmentConfigs</code> on the ReleaseBinding.
Inspect runtime	View K8s Artifacts .	Reads the resource tree from the data plane.

4.3 CLI equivalents (the `occ` commands your project reuses)

```
occ component deploy my-service --namespace default --project default
occ component deploy my-service --release my-service-5d7f658d9c
occ component deploy my-service --to staging
occ component deploy my-service --to production \
  --set spec.componentTypeEnvironmentConfigs.replicas=3
occ releasebinding list --namespace default --project default --component my-
service
occ component get my-service --namespace default
```

The `--set` flag uses dot paths into the ReleaseBinding spec. Example override ReleaseBinding fields: `spec.owner.projectName`, `spec.owner.componentName`, `spec.environment`, `spec.releaseName`, `spec.state`, `spec.componentTypeEnvironmentConfigs`, `spec.traitEnvironmentConfigs`, `spec.workloadOverrides`.

5. Other developer workflows in the portal

- **Create a project.** Create page, template with action `openchoreo:project:create` (inputs: `namespaceName`, `projectName`, `deploymentPipeline`). Projects default to the platform's `default` `ClusterProjectType`.
- **Create a component.** Template with `openchoreo:component:create` (`namespaceName`, `projectName`, `componentName`, `componentType`, plus any parameters the `ComponentType` schema requires). Creates the Component and optional Workload. Components reference types as `{workloadType}/{name}`, for example `deployment/web-service`, usually with `kind: ClusterComponentType`.
- **Create a resource.** `openchoreo:resource:create` (`typeKind`, `typeName` and type parameters).
- **Build from source.** Build tab: pick a Workflow, fill its parameter form (repo URL, build config), trigger a WorkflowRun, watch status and task steps. Build output feeds a new ComponentRelease and, with `autoDeploy`, the first environment.
- **Observe.** Logs, Events, Metrics, Alerts, Traces, Wirelogs tabs; RCA and cost reports at project level.
- **See architecture.** Cell Diagram on the project page.
- **View YAML.** Every portal action has a declarative equivalent; the Definition tab shows the manifest.

6. How the portal is built (what you can change and how)

6.1 Two ways to run it

Setup	Description	How you customize
Shell app (default)	Backstage ships with the <code>openchoreo-control-plane</code> Helm chart. You do not edit Backstage source.	Helm values (<code>backstage.appConfig</code> , <code>backstage.extraEnv</code>), then <code>helm upgrade --reuse-values</code> .
Your own Backstage app	Your workspace with <code>@openchoreo/*</code> plugins installed.	Normal Backstage development.
Fork	Fork <code>openchoreo/backstage-plugins</code> (the in-tree portal).	Full source control.

6.2 Packages

Package	Provides
<code>@openchoreo/backstage-plugin</code>	Core: catalog sync, OIDC sign-in, Domain/System/Component views, Deploy tab (<code>Environments</code>), Cell Diagram, Definition tab, cards, permission policy.
<code>@openchoreo/backstage-plugin-openchoreo-observability (+ -backend)</code>	Logs, Traces, Incidents, RCA, Cost, Events, Metrics, Alerts, Wirelogs tabs.
<code>@openchoreo/backstage-plugin-openchoreo-ci (+ -backend)</code>	Component Build tab.
<code>@openchoreo/backstage-plugin-openchoreo-workflows (+ -backend)</code>	<code>/workflows</code> page and Runs tab.
<code>scaffolder-backend-module-openchoreo</code>	OpenChoreo scaffolder actions.
<code>@openchoreo/backstage-plugin-common</code>	Shared helpers, for example <code>isOpenChoreoComponent</code> .

Tech notes: the plugins use MUI v4 (`@material-ui/core` 4.12.x) and `@material-ui/lab` 4.0.0-alpha.61. Defaults target the **New Frontend System (NFS)**, where tabs and cards auto-mount through extension blueprints; a legacy path with a hand-edited `EntityPage.tsx` is also supported. Check the compatibility matrix for the Backstage release line.

Not packaged for external installation yet: external CI integrations (Jenkins, GitHub Actions, GitLab CI) and the pre-built default scaffolder templates (the actions are packaged, the templates stay in the upstream repo).

6.3 Supported customization hooks

1. **Hide or move a tab (NFS).** Override the extension, for example `entity-content:openchoreo/component-deploy`, with `disabled: true` or a different `attachTo`, via `openchoreoPluginAlphaBase.withOverrides(...)`, then use the customised plugin in `createApp({ features })` instead of `openchoreoPluginAlpha`.
2. **Add or replace cards.** `EntityCardBlueprint` extensions mount into the Overview grid. Extra exports you can opt into: `RuntimeHealthCard`, `DeploymentStatusCard`, `DeploymentPipelineCard`, `EnvironmentStatusSummaryCard`, `AccessControlContent`, `GitSecretsContent`.
3. **Custom scaffolder templates** that chain OpenChoreo actions with built-in ones such as `publish:github:pull-request`. Required: `EXPERIMENTAL_formDecorators: [{ id: openchoreo:inject-user-token }]` in the template spec. This injects the signed-in user's IDP token as the `OPENCHOREO_USER_TOKEN` secret so writes are attributed to the real user.
4. **Custom form fields** (shell app): `NamespaceEntityPicker`, `ResourceNamePicker`, `DeploymentPipelinePicker`, `ComponentNamePicker`, `BuildWorkflowPicker`, `TraitsField` and others, used through `ui:field:`.
5. **Register templates** either through the Catalog Import UI (requires `Template` in global `catalog.rules.allow`) or `catalog.locations` in app-config with a per-location `allow: [Template]` rule. Note Backstage replaces arrays rather than merging, so re-declare the bundled default locations when you override `catalog.locations`.
6. **Permissions.** A composable Backstage permission policy delegates `openchoreo.*` permissions to the OpenChoreo authorization service; everything else falls back to `ALLOW`.

6.4 Available OpenChoreo scaffolder actions

Action	Creates	Audience
<code>openchoreo:namespace:create</code>	Namespace	Platform engineer
<code>openchoreo:project:create</code>	Project	Developer
<code>openchoreo:component:create</code>	Component + optional Workload	Developer
<code>openchoreo:resource:create</code>	Resource	Developer
<code>openchoreo:environment:create</code>	Environment	Platform engineer
<code>openchoreo:deployment-pipeline:create</code>	DeploymentPipeline	Platform engineer
<code>openchoreo:*type-definition:create</code> , <code>*trait-definition:create</code> , <code>*workflow-definition:create</code> (namespaced and cluster-scoped)	ComponentType, ResourceType, Trait, Workflow	Platform engineer (raw YAML input)

There is **no** built-in action that writes Git or opens a PR for a Deploy/Promote/override change. That is the gap Project 702 fills, which matches your gap analysis.

7. Current behaviour vs. what Project 702 needs

Today, portal writes go to the control-plane API and take effect immediately. The Git path exists only through Workflows (build-and-release and bulk promote open PRs). The table below maps each existing developer UI element to the change your project implies. The right-hand column is **[Inference]**: my suggested design, not documented behaviour.

Existing UI element	Today	Project 702 change [Inference]
Deploy tab, Promote button	Calls API; next environment updates immediately.	Launch the <code>openchoreo-gitops-promote</code> template pre-filled with namespace, project, component, source and target environment. Result: a PR, not a live change.
Environment card, gear icon (Overrides)	Writes overrides to the ReleaseBinding through the API.	Keep the same Workload/Component/Traits tabs, but the Save action renders the ReleaseBinding with <code>occ</code> and opens a PR. Add a note that the change is pending review.
Environment card status (Ready/NotReady/Failed)	Live binding condition only.	Add a lifecycle chip before it: PR Open, In review, Merged, Synced, then Ready or Blocked. This is your status-chip PoC.
Create page, component/project templates	Call <code>openchoreo*:create</code> actions (API writes).	Optional GitOps variants that emit manifests and open a PR; the promotion template is the MVP.
Overview tab cards	Deployment status, runtime health.	Add a "Pending changes" card listing open PRs for the component, with author, state and link.
Deploy tab, conflict situation	Not handled.	Before submitting, call the conflict-check action and show the overlapping PR with view, update or cancel choices.
Definition tab	Shows live manifest.	Optionally show the Git-side manifest and diff against the open PR.

7.1 Where to integrate, in order of least invasive first

1. **Templates only (no UI fork).** Ship the promotion template, add the template to the catalog, and link to it from the component page. Works on the shell app with Helm values alone.
2. **Add cards and a tab via NFS extensions.** New `EntityCardBlueprint` (pending changes, status chip) and `EntityContentBlueprint` extensions in your own Backstage app or a small plugin. Disable or wrap the stock Deploy extension if needed.
3. **Replace the Promote and Overrides behaviour inside Deploy.** The `Environments` component is part of `@openchoreo/backstage-plugin`, so changing button behaviour means forking `openchoreo/backstage-plugins` or wrapping the component. This is the highest-maintenance option because it must track upstream releases.

7.2 Constraints to keep in mind

- Mixed mode: if Flux owns a resource, a portal API write will be reverted on the next sync. Your design should stop the direct Promote/Overrides API path for GitOps-managed projects.
- Every OpenChoreo action needs the user token decorator when auth is on.
- Entities must carry OpenChoreo annotations for the Deploy tab to work.
- Plugin versions must match the Backstage release line (see the compatibility matrix).
- The two project files you shared (the dark slide deck and the interactive proposal) are standalone HTML mockups, not the portal itself. They are a good reference for the *target* design language; the real portal uses Backstage with MUI v4 theming, so a visual redesign should be done with a Backstage theme (and extension overrides) rather than by replacing the HTML.

8. Suggested next steps for the UI work

1. Run the quick-start locally and walk the Deploy tab, Overrides page and Create page once; screenshot each as your baseline.
2. Decide approach 1, 2 or 3 above (I would start with 1, add 2 for the status chip, and treat 3 as post-MVP).
3. Read the `Environments` source in `openchoreo/backstage-plugins` to confirm how Promote and Overrides save, and which hooks you can wrap. I have not inspected that source.

4. Confirm with your mentor whether PR creation lives in Backstage or in the control-plane API (your open question), since it changes what the UI calls.

9. Source pages used

- What is OpenChoreo, Developer Abstractions, Deploy and Promote (v1.3.x)
- Environment Overrides (the cached copy I read was the v1.2.x version; confirm against v1.3.x)
- Backstage Plugins overview and Entity views (v1.3.x)
- Custom Backstage Scaffolders Templates (v1.3.x)
- Explore: Backstage-Powered Developer Portal
- Your uploaded gitops-pr-driven.md, scaffolder.yaml and proposal HTML files, for the gap analysis and template design

Further pages worth reading: Project Releases, Logs and Troubleshooting, Installing into an existing Backstage app, Catalog sync, Permission policy and the compatibility matrix, all under openchoreo.dev/docs/.